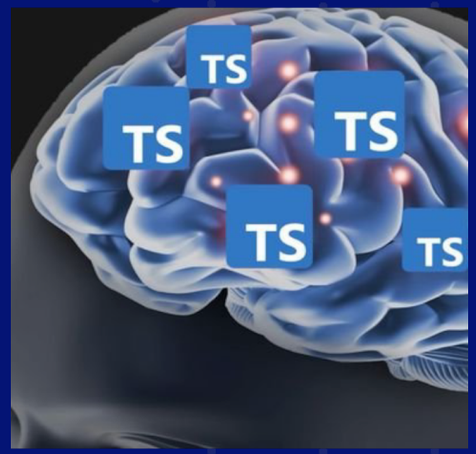
TypeScript's Type Wizardry

(and where to use it)



Fanis Tharropoulos



Who Am I?

Software Engineer at typesense

Some of my own projects

museslabs/**phonto**

Video wallpaper program for wayland compositors and macos

Rust ☆ 436 ⑂ 11

museslabs/**stochos**

Keyboard driven mouse control for Linux and macOS

Rust ☆ 148 ⑂ 17

typesense-ts

A fully typesafe Typesense client for typeheads

TypeScript ☆ 53 ⑂ 7

museslabs/**kyma**

Presentations from markdown in the terminal with fancy transition animations

Go ☆ 98 ⑂ 2

What I work on day to day

typesense/**typesense**

A Fast, typo tolerant, in-memory fuzzy Search Engine for building delightful search experiences

C++ ☆ 27k ⑂ 1k

typesense/*

I work on all of the official API clients and integrations

JS/TS, Python, Go, PHP, Ruby, Java, Dart...



fanis.iusevimbtw.com

tharropoulos

The compiler read my string

TYPESCRIPT / a typesense-ts search

```ts
const users = collection({
  name: "users",
  fields: [
    { name: "name", type: "string" },
    { name: "age", type: "int32", sort: true },
    { name: "email", type: "string", optional: true },
  ],
});

await users.search({
  q: "fanis",
  query_by: ["name"],
  sort_by: "email:desc",
});
```

TSC / the error, wrapped to fit

```
error TS2345: Argument of type
  '{ q: string; query_by: string[]; sort_by:
string; }'
is not assignable to parameter of type
  '"[Error on sort_by]: Invalid token sequence:
    Invalid identifier: email is not a sortable
field."'
```

No request sent, no test run. Just the editor.

By the end of this talk, you'll know how this works.



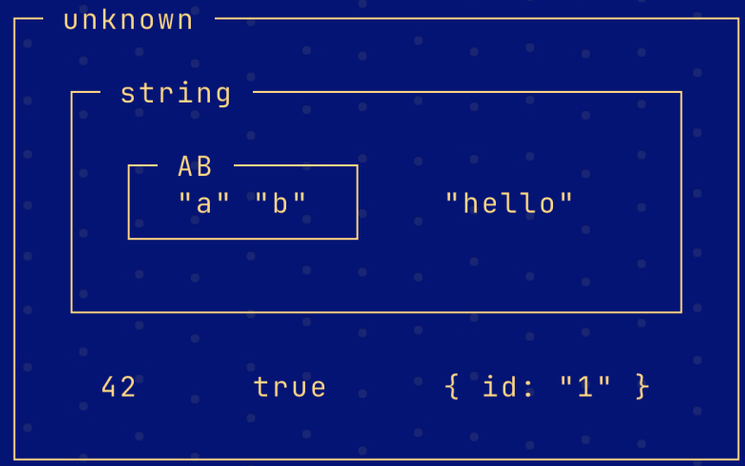
Types are sets

TYPESCRIPT / types as sets of values

```ts
type AB = "a" | "b";  // two values
type S  = string;     // every string
type U  = unknown;    // every value
type N  = never;      // no values at all
type A  = any;        // not a set: checking off

// does the left side fit inside the right?
type T1 = "a" extends AB ? true : false;     // true
type T2 = AB extends string ? true : false;  // true
type T3 = string extends AB ? true : false;  // false
type T4 = any extends AB ? true : false;     // boolean
```

TEXT / the same types, drawn



never: the empty set
any: no box, it opts out

`extends` **asks one question: does the left side fit inside the right?**



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

Ideas you already use, one level up. The rest of the talk fills in the right column.



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

if becomes a conditional type that asks whether `X` fits inside `string` or not.



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

Destructuring becomes `infer` to match the shape and name the pieces.



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

A loop over keys becomes a mapped type: one property out for every key in.



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

Types have no loops. A type that calls itself is the loop.



The type system is a small language

THE IDEA

if
destructure
loop over keys
recursion
match a string

JAVASCRIPT / values

```
typeof x === "string" ? 1 : 0
const [head, ...tail] = list
for (const key in obj) out[key] = obj[key]
const f = (x) => Array.isArray(x) ? f(x[0]) :
x
s.startsWith("gid://") ? s.slice(6) : null
```

TYPESCRIPT / types

```
X extends string ? 1 : 0
L extends [infer Head, ...infer Tail] ? Head : never
{ [Key in keyof Obj]: Obj[Key] }
type F<X> = X extends (infer I)[] ? F<I> : X
S extends `gid://${infer Id}` ? Id : null
```

Matching a string becomes a template literal type: match a pattern, `infer` the rest.



Where did my autocomplete go?

TYPESCRIPT / a search function

```ts
function search(q: "*" | string) {}

search("");
//      ^ autocomplete suggests nothing

// hover over q:
// (parameter) q: string
```

We asked for `"*" | string` but got `string` back. Where did the wildcard go?



Where did my autocomplete go?

TYPESCRIPT / the fix

```ts
// "*" fits inside string, so the union collapses
type Q1 = "*" | string;          // string

// string & {} is still every string,
// but TypeScript won't merge "*" into it
type Q2 = "*" | (string & {});   // "*" | (string & {})
```

TYPESCRIPT / typesense-ts, src/search.ts, trimmed

```ts
interface QueryParams<
  Fields extends CollectionField[],
  Q extends "*" | (string & {}),
> {
  q: Q;
  query_by: QueryBy<Fields>;
}
```

`string & {}` holds every string, but TypeScript never merges it with `"*"` so the wildcard stays.



Mapped types are loops

TYPESCRIPT / a mapped type, and what it becomes

```ts
type Resource = "products" | "collections" | "articles";

// for every R in Resource, write one property
type Schemas = { [R in Resource]: Schema };

// Schemas is now:
// {
//   products: Schema;
//   collections: Schema;
//   articles: Schema;
// }
```

TYPESCRIPT / a tool I'm building: forget one

```ts
const schemas: Schemas = {
  products: productsSchema,
  collections: collectionsSchema,
};

// Property 'articles' is missing in type
//   '{ products: Schema; collections: Schema; }'
//   but required in type 'Schemas'.
```

One property for every member of the union. Forget one, and it doesn't compile.



Parsing strings with infer

TYPESCRIPT / a sort_by parser, in miniature

```ts
type ParseSort<S> =
  S extends `${infer Field}:${infer Dir}` ? { field: Field; dir: Dir } : never;

type P = ParseSort<"age:desc">;   // { field: "age"; dir: "desc" }

type Sortable = "age" | "score";
type CheckSort<S> =
  S extends `${infer Field}:${"asc" | "desc"}`
    ? Field extends Sortable ? S : `${Field} is not a sortable field.`
    : "Expected field:asc or field:desc";

declare function search<const S extends string>(
  params: { sort_by: CheckSort<S> },
): void;

search({ sort_by: "email:desc" });
// Type '"email:desc"' is not assignable
//   to type '"email is not a sortable field."'.
```

`infer` inside a template literal splits a string wherever the pattern says.



Parsing strings with infer

TYPESCRIPT / a sort_by parser, in miniature

```ts
type ParseSort<S> =
  S extends `${infer Field}:${infer Dir}` ? { field: Field; dir: Dir } : never;

type P = ParseSort<"age:desc">;   // { field: "age"; dir: "desc" }

type Sortable = "age" | "score";
type CheckSort<S> =
  S extends `${infer Field}:${"asc" | "desc"}`
    ? Field extends Sortable ? S : `${Field} is not a sortable field.`
    : "Expected field:asc or field:desc";

declare function search<const S extends string>(
  params: { sort_by: CheckSort<S> },
): void;

search({ sort_by: "email:desc" });
// Type '"email:desc"' is not assignable
//   to type '"email is not a sortable field."'.
```

Check the pieces against the schema. The answer can be an error message.



Parsing strings with infer

TYPESCRIPT / a sort_by parser, in miniature

```
type ParseSort<S> =
  S extends `${infer Field}:${infer Dir}` ? { field: Field; dir: Dir } : never;

type P = ParseSort<"age:desc">;   // { field: "age"; dir: "desc" }

type Sortable = "age" | "score";
type CheckSort<S> =
  S extends `${infer Field}:${"asc" | "desc"}`
    ? Field extends Sortable ? S : `${Field} is not a sortable field.`
    : "Expected field:asc or field:desc";

declare function search<const S extends string>(
  params: { sort_by: CheckSort<S> },
): void;

search({ sort_by: "email:desc" });
// Type '"email:desc"' is not assignable
//   to type '"email is not a sortable field."'.
```

Make it the parameter's type, and the error shows up where you call it.



Parsing strings with infer

TYPESCRIPT / typesense-ts, src/lexer/types.ts, trimmed

```ts
// read an identifier, one character at a time
type ReadString<T, Acc extends string = ""> =
  T extends `${infer Head}${infer Tail}`
    ? Head extends Char
      ? ReadString<Tail, `${Acc}${Head}`>
      : [Acc, T]
    : [Acc, T];
```

TYPESCRIPT / typesense-ts, tests/sort.test.ts

```ts
expectTypeOf<SortTokenizer<"age:desc">>().toEqualTypeOf<
  [Ident<"age", FieldType>, Colon, "desc"]
>();
```

typesense-ts reads `sort_by` one character at a time, then checks every token against your schema.



Why do I need recursion?

TYPESCRIPT / loops and counting

```ts
// no for, no while: a type that calls itself is the loop
type TupleOf<L extends number, T, Acc extends T[] = []> =
  Acc["length"] extends L ? Acc : TupleOf<L, T, [...Acc, T]>;

type Three = TupleOf<3, number>;   // [number, number, number]

// a tuple's length is a number literal, so you can count
type AddOne<N extends number> = [...TupleOf<N, 1>, 1]["length"];
type Four = AddOne<3>;   // 4
```

Types have no loops. Recursion is the loop, and tuple lengths are how you count.



Why do I need recursion?

TYPESCRIPT / typesense-ts

```ts
// query_by_weights?: TupleOfLength<L, number>
// where L is the length of query_by

await books.search({
  q: "dune",
  query_by: ["title", "authors", "categories"],
  query_by_weights: [3],
});
```

TSC / the error

```
error TS2322: Type '[number]' is not assignable
  to type '[number, number, number]'.
  Source has 1 element(s) but target requires 3.
```

One weight per `query_by` field, checked by counting at the type level.



TypeScript types can run DOOM

TEXT / Dimitri Mitropoulos, launch video, February 2025

3.5 trillion	lines of types
177 TB	of types, through the type checker
12 days	around the clock, for the first frame
20 million	type instantiations a second
	(a whole app's check: 200,000 to 500,000)
100 GB	of memory for the type checker

TEXT / how Doom got in

DOOM, in C
↓ compiled to WebAssembly
↓ rewritten as types
doom.ts: 99,342 lines of types
↓ run by a computer made of types
one frame, as an object type
that you hover over in the editor

Every instruction of the game ran in the type checker.



A computer made of types

TYPESCRIPT / a program

```ts
type Execution = executeInstruction<{
  instructions: [
    { kind: 'add', value: 'hello' },
    { kind: 'add', value: ' world' },
    { kind: 'uppercase' },
    { kind: 'add', value: '!' },
  ];
  result: '';
}>;
// "HELLO WORLD!"
```

TYPESCRIPT / the simplest machine in the repo, trimmed

```ts
type executeInstruction<
  state extends ProgramState
> =
  state['instructions'] extends [
    infer instruction extends Instruction,
    ...infer remainingInstructions extends Instruction[]
  ]
  ? executeInstruction<
      selectInstruction<state, remainingInstructions, instruction>
    >
  : state['result'];
```

Take the next instruction, run it, call yourself. Destructure and recursion, from the table.



Adding with tuples

TYPESCRIPT / ts-type-math/add.ts, trimmed

```ts
type DigitToCounter<T extends string> =
  T extends '0'
  ? []
  : [1]

type Calculate<T extends Counter> =
  T['length'] extends 0 ? { digit: '0', carry: [] } :
  T['length'] extends 1 ? { digit: '1', carry: [] } :
  T['length'] extends 2 ? { digit: '0', carry: [1] } :
  T['length'] extends 3 ? { digit: '1', carry: [1] } :
  never
```

TEXT / 1 + 1, with a carry of 1

```
'1'  +  '1'  +  carry
[1]  +  [1]  +  [1]

→ [1, 1, 1]
→ length 3
→ { digit: '1', carry: [1] }
```

Doom adds like `AddOne` did: put 1s in a tuple, then read its length.



TypeScript 7: the compiler, rewritten in Go

TEXT / full build, TypeScript 6 → 7

vscode	125.7s	→	10.6s	11.9x
sentry	139.8s	→	15.7s	8.9x
bluesky	24.3s	→	2.8s	8.7x
playwright	12.8s	→	1.47s	8.7x
tldraw	11.2s	→	1.46s	7.7x

The same type system, about 10x faster.



TypeScript 7.1: content mappers

TEXT / what a content mapper does

```
before   tsc checks .ts and .js files, nothing else
         a .vue, .svelte or .sql import is a type
         you write by hand

7.1      a mapper turns the file into TypeScript
         tsc checks it, and reports errors
         in the original file
```

JSONC / tsconfig.json, trimmed

```jsonc
{
  "contentMappers": [
    { "package": "reylo", "extensions": [".sql"] }
  ]
}

$ tsc --noEmit --runExternalCode
```

A content mapper lets tsc check files that aren't TypeScript, and point errors back at them.



Reylo: SQL files, typed by your database

SQL / get_user.sql

```sql
select id, name, email
from users
where id = :id::int
```

TYPESCRIPT / main.ts, from Reylo's docs

```ts
import getUser from "./get_user.sql";

const user = await getUser.run(db, { id: 1 });
//    ^? const user: {
//          id: number;
//          name: string;
//          email: string | null;
//       } | null
```

Import a .sql file, and its types come from your database.



Reylo: a typo in the SQL is a type error

SQL / with a typo

```
select id, nmae, email
from users
where id = :id::int
```

TSC / from Reylo's docs

```
$ tsc --noEmit --runExternalCode
get_user.sql:1:12 - error reylo6: column "nmae" does not exist (42703)

1 select id, nmae, email
             ~~~~

Found 1 error in get_user.sql:1
```

The database checks the query, and tsc reports it in the .sql file.



So where should you use it?

IN APP CODE / a little goes a long way

`"*" | (string & {})`	suggest values, take any string
mapped records	cover every member of a union
content mappers	let a tool type .sql and .vue

IN A LIBRARY / spend it at the boundary

derive types from the schema users give you
check the strings users write
put the fix in the error message
test the types like the rest of the API

WHAT IT COSTS

compile time	Doom took 12 days. Measure yours: tsc --generateTrace
error messages	a typo in a filter can come back as "Unknown token: !"
maintenance	someone has to read your types next year

Use the least wizardry that makes the wrong call impossible.



Q&A · 28 / 28

Thank you. Questions?

LEARN MORE / keep practicing

TypeHero	typehero.dev
Type Challenges	github.com/type-challenges/type-challenges
TypeScript Handbook	typescriptlang.org/docs/handbook/2/types-from-types.html
Doom in types	youtu.be/0mCsluv5FXA
Reylo	reylo.dev

$ whoami

fanis@fosscomm

Name: Fanis Tharropoulos
Work: Typesense
GitHub: github.com/tharropoulos
Site: fanis.iusevimbtw.com
Library: typesense-ts
Slides: made with Kyma
Status: taking questions

Ask me anything, the weirder the type the better.

